

In [1]: `!pip install numpy pandas seaborn`

```
Requirement already satisfied: numpy in c:\users\michael\anaconda3a\lib\site-packages (1.26.4)
Requirement already satisfied: pandas in c:\users\michael\anaconda3a\lib\site-packages (2.2.3)
Requirement already satisfied: seaborn in c:\users\michael\anaconda3a\lib\site-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2023.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\michael\anaconda3a\lib\site-packages (from seaborn) (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\michael\anaconda3a\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\michael\anaconda3a\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\michael\anaconda3a\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\michael\anaconda3a\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\michael\anaconda3a\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\michael\anaconda3a\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\michael\anaconda3a\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)
Requirement already satisfied: six>=1.5 in c:\users\michael\anaconda3a\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
```

In [2]: `# pandas is a software library written for the Python programming language for
import pandas as pd
#NumPy is a library for the Python programming language, adding support for large
import numpy as np
Matplotlib is a plotting library for python and pyplot gives us a MatLab Like
import matplotlib.pyplot as plt
#Seaborn is a Python data visualization library based on matplotlib. It provides
import seaborn as sns`

In [4]: `# Load the CSV file from URL
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS
df = pd.read_csv(url)

Display the first 5 rows of the dataframe
print("First 5 rows of the dataset:")
df.head()`

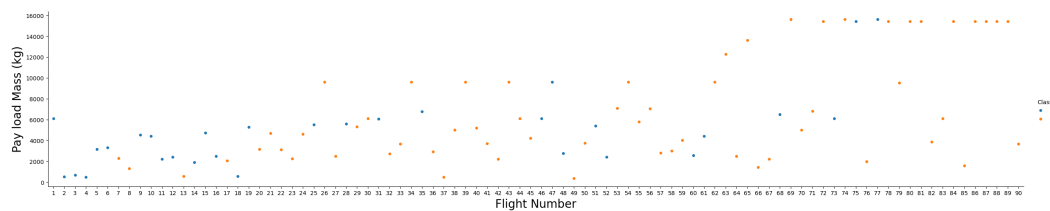
First 5 rows of the dataset:

Out[4]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcom
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	Nor Nor
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	Nor Nor
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	Nor Nor
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	Fal Oce
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	Nor Nor

In [5]:

```
sns.catplot(y="PayloadMass", x="FlightNumber", hue="Class", data=df, aspect = 5)
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Pay load Mass (kg)", fontsize=20)
plt.show()
```



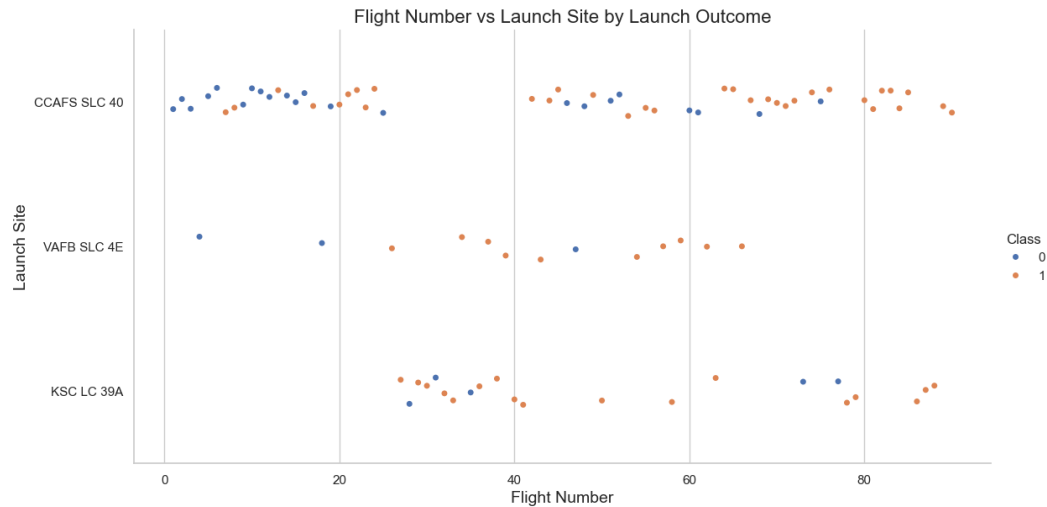
In [6]:

```
# Import Libraries (if not already done)
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

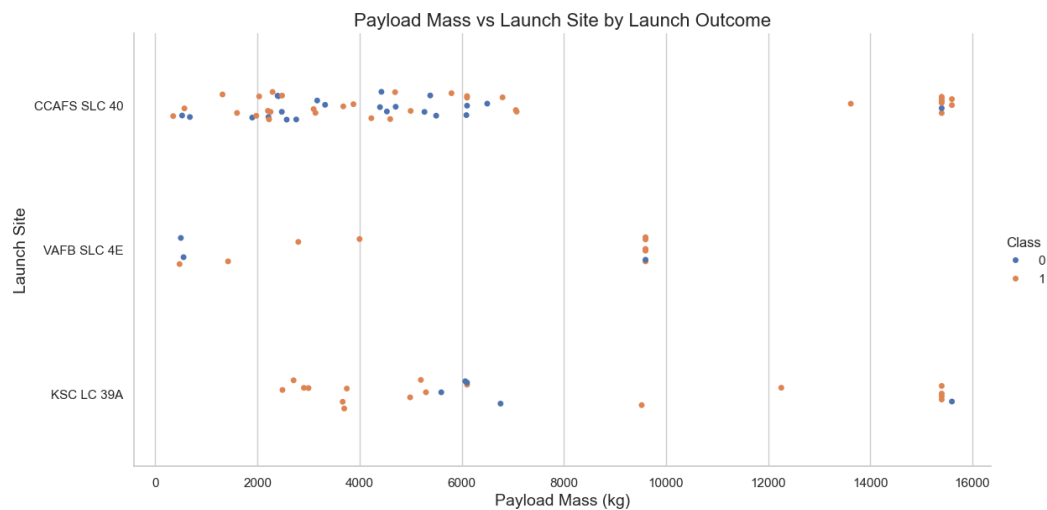
# Load the dataset
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DSE101-Data/SpaceData.csv"
df = pd.read_csv(url)

# Create the catplot
sns.set(style="whitegrid")
sns.catplot(
    data=df,
    x="FlightNumber",
    y="LaunchSite",
    hue="Class",
    kind="strip", # scatter plot style
    height=6,
    aspect=2
)

# Add Labels and title
plt.xlabel("Flight Number", fontsize=14)
plt.ylabel("Launch Site", fontsize=14)
plt.title("Flight Number vs Launch Site by Launch Outcome", fontsize=16)
plt.show()
```



In [7]:



In [9]:

```
# Import libraries (if not already done)
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DSE101-Data-sets/airline.csv"
df = pd.read_csv(url)

# Calculate success rate per orbit
orbit_success = df.groupby("Orbit")["Class"].mean().reset_index()

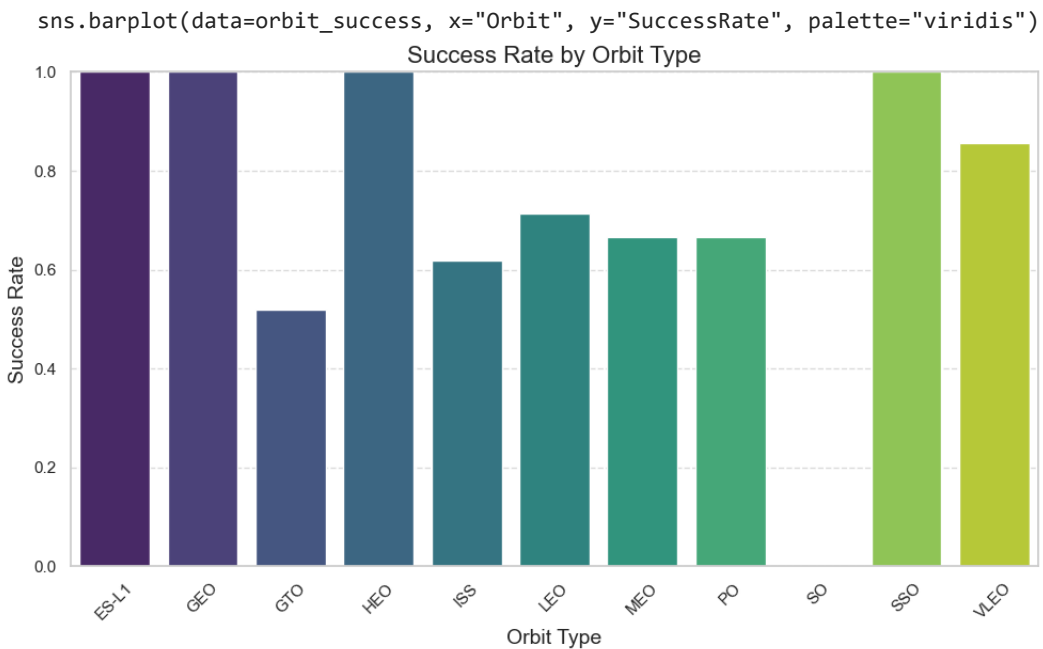
# Rename the column for clarity
orbit_success.columns = ["Orbit", "SuccessRate"]

# Plot the bar chart
plt.figure(figsize=(10, 6))
sns.barplot(data=orbit_success, x="Orbit", y="SuccessRate", palette="viridis")
```

```
# Add Labels and title
plt.xlabel("Orbit Type", fontsize=14)
plt.ylabel("Success Rate", fontsize=14)
plt.title("Success Rate by Orbit Type", fontsize=16)
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

C:\Users\Michael\AppData\Local\Temp\ipykernel_15256\2140404772.py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



```
In [10]: # Import Libraries (if not already done)
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS
df = pd.read_csv(url)

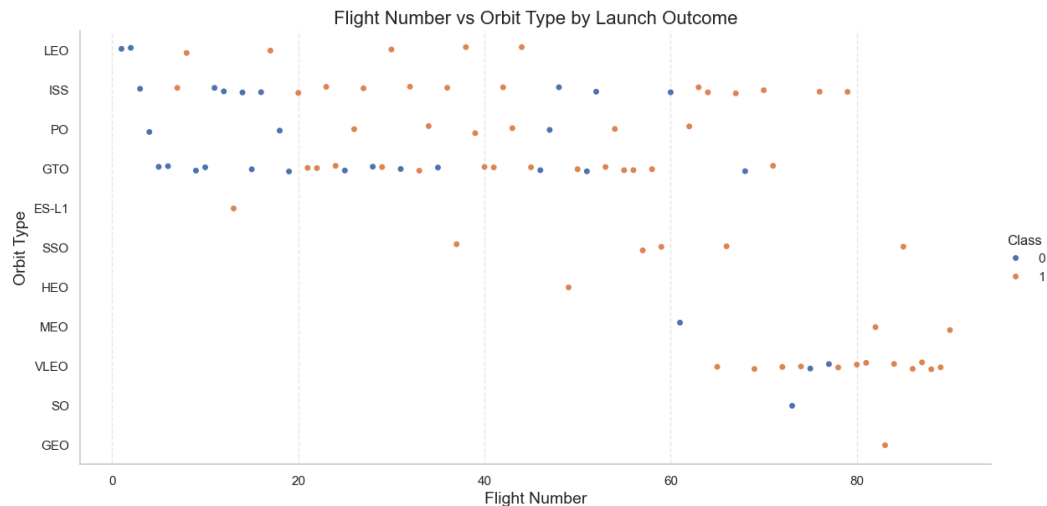
# Create the scatter plot using Seaborn's catplot
sns.set(style="whitegrid")
sns.catplot(
    data=df,
    x="FlightNumber",
    y="Orbit",
    hue="Class",          # 0 = Failure, 1 = Success
    kind="strip",
```

```

        height=6,
        aspect=2
    )

    # Customize the plot
    plt.xlabel("Flight Number", fontsize=14)
    plt.ylabel("Orbit Type", fontsize=14)
    plt.title("Flight Number vs Orbit Type by Launch Outcome", fontsize=16)
    plt.grid(axis='x', linestyle='--', alpha=0.5)
    plt.show()

```



```

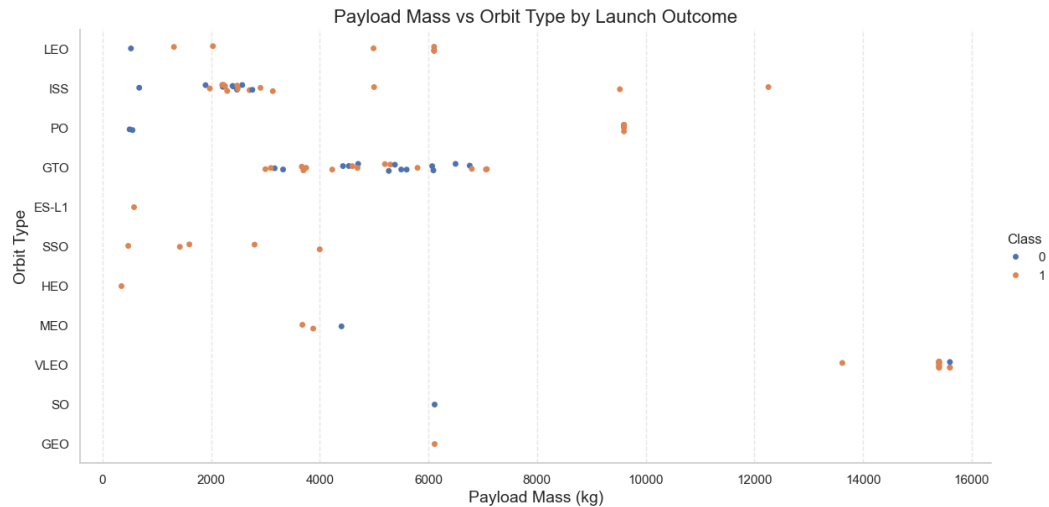
In [1]: # Import libraries (if not already done)
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS
df = pd.read_csv(url)

# Create the scatter-style catplot
sns.set(style="whitegrid")
sns.catplot(
    data=df,
    x="PayloadMass",
    y="Orbit",
    hue="Class",          # 0 = Failure, 1 = Success
    kind="strip",
    height=6,
    aspect=2
)

# Customize the plot
plt.xlabel("Payload Mass (kg)", fontsize=14)
plt.ylabel("Orbit Type", fontsize=14)
plt.title("Payload Mass vs Orbit Type by Launch Outcome", fontsize=16)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.show()

```



```
In [2]: # Import Libraries (if not already done)
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DSE10-Notebooks/Course_1/EDA/EDA_data/launch_outcome.csv"
df = pd.read_csv(url)

# Function to extract year from date
def Extract_year():
    return [i.split("-")[0] for i in df["Date"]]

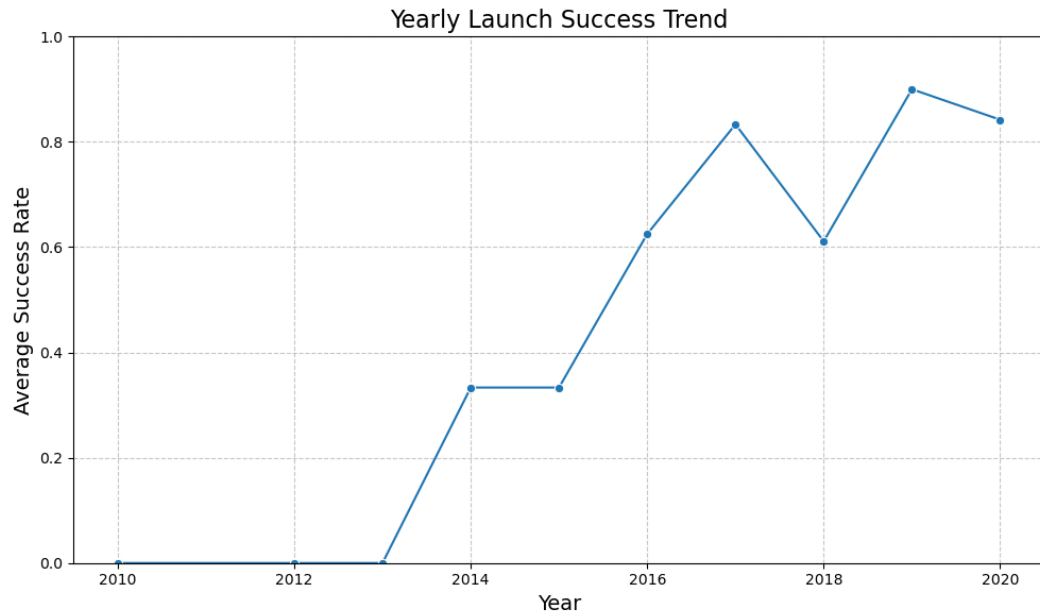
# Apply function and update the 'Date' column to only have year
df['Year'] = Extract_year()

# Convert Year to int type for plotting
df['Year'] = df['Year'].astype(int)

# Group by year and calculate average success rate (mean of 'Class')
yearly_success = df.groupby('Year')['Class'].mean().reset_index()

# Plot the trend
plt.figure(figsize=(10,6))
sns.lineplot(data=yearly_success, x='Year', y='Class', marker='o')

# Customize plot
plt.xlabel("Year", fontsize=14)
plt.ylabel("Average Success Rate", fontsize=14)
plt.title("Yearly Launch Success Trend", fontsize=16)
plt.ylim(0, 1)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



```
In [4]: # Select relevant features
features = df[['FlightNumber', 'PayloadMass', 'Orbit', 'LaunchSite', 'Flights',
              'GridFins', 'Reused', 'Legs', 'LandingPad', 'Block',
              'ReusedCount', 'Serial']]

# Apply one-hot encoding to the categorical columns
features_one_hot = pd.get_dummies(features,
                                   columns=['Orbit', 'LaunchSite', 'LandingPad',

# Display the first few rows
features_one_hot.head()
```

```
Out[4]:
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	ReusedCount
0	1	6104.959412	1	False	False	False	1.0	0
1	2	525.000000	1	False	False	False	1.0	0
2	3	677.000000	1	False	False	False	1.0	0
3	4	500.000000	1	False	False	False	1.0	0
4	5	3170.000000	1	False	False	False	1.0	0

5 rows × 80 columns

```
In [5]: # Cast the entire dataframe to float64
features_one_hot = features_one_hot.astype('float64')

# Optional: Check the data types to confirm
features_one_hot.dtypes.head()
```

```
Out[5]: FlightNumber    float64
PayloadMass    float64
Flights        float64
GridFins       float64
Reused         float64
dtype: object
```

```
In [6]: # Export the processed DataFrame to a CSV file
features_one_hot.to_csv('dataset_part_3.csv', index=False)
```

```
In [7]: # Select the original features
features = df[['FlightNumber', 'PayloadMass', 'Orbit', 'LaunchSite', 'Flights',
              'GridFins', 'Reused', 'Legs', 'LandingPad', 'Block',
              'ReusedCount', 'Serial']]

# Apply one-hot encoding
features_one_hot = pd.get_dummies(features, columns=['Orbit', 'LaunchSite', 'La

# Get the total number of columns
print("Total columns after one-hot encoding:", features_one_hot.shape[1])
```

Total columns after one-hot encoding: 80

```
In [3]: # Identify key points
first_year = yearly_success.iloc[0]
peak_success = yearly_success.loc[yearly_success['Class'].idxmax()]
lowest_success = yearly_success.loc[yearly_success['Class'].idxmin()]

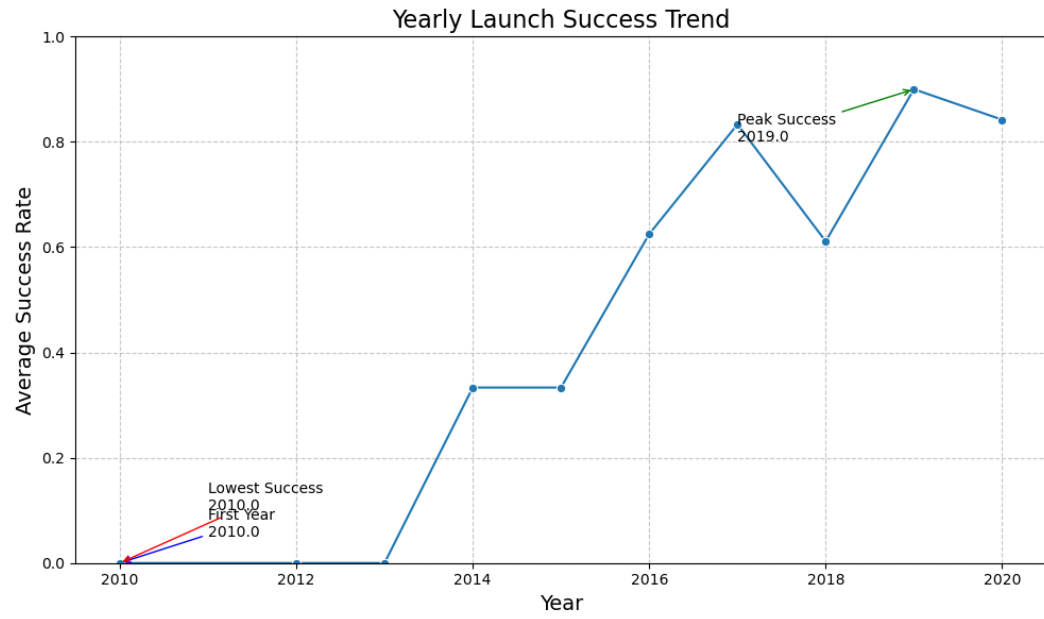
# Plot the trend
plt.figure(figsize=(10, 6))
sns.lineplot(data=yearly_success, x='Year', y='Class', marker='o')

# Add annotations
plt.annotate(f"First Year\n{first_year['Year']}",
            xy=(first_year['Year'], first_year['Class']),
            xytext=(first_year['Year'] + 1, first_year['Class'] + 0.05),
            arrowprops=dict(arrowstyle="->", color='blue'), fontsize=10)

plt.annotate(f"Peak Success\n{peak_success['Year']}",
            xy=(peak_success['Year'], peak_success['Class']),
            xytext=(peak_success['Year'] - 2, peak_success['Class'] - 0.1),
            arrowprops=dict(arrowstyle="->", color='green'), fontsize=10)

plt.annotate(f"Lowest Success\n{lowest_success['Year']}",
            xy=(lowest_success['Year'], lowest_success['Class']),
            xytext=(lowest_success['Year'] + 1, lowest_success['Class'] + 0.1),
            arrowprops=dict(arrowstyle="->", color='red'), fontsize=10)

# Customize plot
plt.xlabel("Year", fontsize=14)
plt.ylabel("Average Success Rate", fontsize=14)
plt.title("Yearly Launch Success Trend", fontsize=16)
plt.ylim(0, 1)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

In []: